

Suppose you are given a workspace with a single axis-parallel regular obstacle with lower left corner $r^- = (x^-, y^-)$ and upper right corner $r^+ = (x^+, y^+)$. You may assume the rectangle lies above the x -axis, but it may lie either side of (or overlap) the y -axis. Describe the shape of the resulting C-obstacle in the (w, h) configuration space of the crane.

Solution: There are three cases:

1. The rectangle is to the right of the y -axis ($x^- > 0$).

In this case, we can break the C-obstacle into three components. The first component will be denoted by r_1 and will be identical to r but transferred to the (w, h) space. Clearly $p_2 = (w, h)$ will intersect with r for any (w, h) within r_1 .

The second component will have corners $r_2^- = (x^-, y^+)$ and $r_2^+ = (x^+, y^+ + 1)$. This component ensures that the height of p_3 always exceeds the height of r while the crane is over r .

The third component covers the case where the tip of the crane is to the right of r and the crane can descend by one unit because it is no longer possible for $\overline{p_2 p_3}$ to intersect r . We will still have to consider whether or not $\overline{p_1 p_2}$ intersects r . The component will be another rectangle which will extend out infinitely into the w -axis. The corners are defined as $r_3^- = (x^+, y^-)$ and $r_3^+ = (\infty, y^+)$.

These three components will form the C-obstacle in the (w, h) configuration space of the crane.

2. The rectangle is to the left of the y -axis ($x^+ < 0$).

In this case, there will be no C-obstacle. Since $h \geq 0$ and $w \geq 0$, the crane cannot enter the space to its left. Therefore, the crane will be able to move throughout its configuration space unimpeded.

3. The rectangle overlaps the y -axis ($x^- \leq 0 \leq x^+$).

In this case, the C-obstacle will be the halfplane $h \geq y^-$ that prevents the crane from moving up into r . The crane will move throughout the w -axis unimpeded.

■

- (a) Let S be a set of n axis-parallel rectangles in the plane. We want to be able to report all rectangles in S that are completely contained in a query rectangle $Q = [x_{lo}, x_{hi}] \times [y_{lo}, y_{hi}]$. Describe a data structure for this problem that uses $O(n \log^3 n)$ space and has $O(\log^3 n + k)$ query time, where k is the number of reported rectangles.

Solution: To solve this problem we will use a 4d orthogonal range tree and cascading search. We will first build the tree on the set of rectangles S . The dimensions for our tree will be the four boundaries, $x_{lo}, x_{hi}, y_{lo}, y_{hi}$, where (x_{lo}, y_{lo}) is the lower left and (x_{hi}, y_{hi}) is the upper right corner of a rectangle in S . As long as these two points are contained within the query rectangle, the whole rectangle is contained within the query rectangle.

Construction.

1. Construct a 1-dimensional range tree (balanced binary search tree) such that the leaves of the tree are sorted by x_{lo} coordinate. Each node u in this tree is associated with a canonical subset $S(u) \subseteq S$ of elements of S that are in the leaves descended from u .
2. For each node u in the above tree, construct an auxiliary tree (1-dimensional range tree) for the rectangles in $S(u)$, sorted by x_{hi} coordinate.
3. For each node u in every auxiliary tree, add an auxiliary tree for $S(u)$, sorted by y_{lo} coordinate.
4. Repeat the above step for the y_{hi} coordinate.

Querying. To execute a query on this tree, we search the initial tree for every node for which all x_{lo} coordinates are bounded by $[x_{lo}, x_{hi}]$ from the query rectangle. From each of these nodes, we descend to its auxiliary tree in the next level and then repeat this process for x_{hi} , y_{lo} , and finally y_{hi} (using $[y_{lo}, y_{hi}]$ from the query rectangle for the y boundaries). At the final level, we will have a list of rectangles that are completely contained within the query rectangle.

Our current query time would be $O(\log n)$ per layer. With four dimensions and k reported rectangles, we have $O(\log^4 n + k)$. To improve this, we use cascading search. For the third set of auxiliary trees, we will add auxiliary lists to each node. Consider node v with children v' and v'' . v 's list will consist of the rectangles from v' and v'' sorted by their y_{hi} coordinate. The elements in this list will store a pointer to the first element in the lists of v' and v'' that has a y_{hi} coordinate greater than or equal to the y_{hi} coordinate of the element in v . With this structure we can drop down and traverse the final level in $O(1)$ time, thus removing a factor of $\log n$ from the query. Therefore, the query time is $O(\log^3 n + k)$.

Space Complexity. The space complexity of the initial tree is $O(n)$. Each auxiliary tree will contain $|S(u)|$ rectangles and, since each rectangle appears in every ancestor of its leaf, every rectangle will appear in $O(\log n)$ auxiliary trees. Therefore, each rectangle contributes $O(\log n)$ to the total space complexity. Since there are $O(n)$ rectangles, the complexity of just the initial tree and the first auxiliary trees is $O(n \log n)$. Each subsequent level of auxiliary trees will multiply another $O(\log n)$ to the space complexity. So at four levels, the total space complexity of the tree is $O(n \log^3 n)$.

■

- (b) Let P be a set of n points in the plane. We want to be able to report all points in P that are completely contained in a query triangle. Fortunately, the triangle is guaranteed to have one horizontal edge, one vertical edge, and one edge of slope -1 or $+1$. Describe a data structure for this problem that uses $O(n \log^3 n)$ space and has $O(\log^3 n + k)$ query time, where k is the number of reported points.

Solution: We will use the same process as in the previous problem, but with a few modifications. We will only have 3 dimensions: x , y , and z . Using the x and y values of each point in P , we construct a 2-dimensional range tree and use the x_{lo} , x_{hi} , y_{lo} , and y_{hi} values of the query triangle to bound our queries.

For z we will have to calculate two values for each point before constructing the auxiliary trees. These values will be the distance from each point to the line $y = x$ if the slope of the query triangle is 1, or $y = -x$ if the slope of the query triangle is -1 . They will be denoted by z^+ and z^- . With these distances, we can establish whether or not the point is within the bounds of the diagonal edge of the query triangle and its opposite corner.

These values are calculated using the point to line distance formula:

$$d(p, l) = \frac{ax_i + by_i + c}{\sqrt{a^2 + b^2}}$$

$$p_i = (x_i, y_i) \in P$$

$$l = \{ax + by + c = 0\}$$

$$\begin{aligned} z^+ &= \{d(p, l) \mid l = \{y = x\}\} \\ &= \frac{x_i - y_i + 0}{\sqrt{1^2 + (-1)^2}} \\ &= \frac{x_i - y_i}{\sqrt{2}} \end{aligned}$$

$$\begin{aligned} z^- &= \{d(p, l) \mid l = \{y = -x\}\} \\ &= \frac{x_i + y_i + 0}{\sqrt{1^2 + 1^2}} \\ &= \frac{x_i + y_i}{\sqrt{2}} \end{aligned}$$

With these values, we can construct the final set of auxiliary trees. For every node in the y -value auxiliary trees, we will have to add two auxiliary trees, one for z^+ and one for z^- . In addition, we will have to calculate the z values for the query triangle. We can simply calculate z^+ or z^- for one of the points on the diagonal edge (the value will be the same for both points on the diagonal edge) and for the third point. All points within the triangle must fall between these two values on the z -axis.

Given the same logic for space and time complexity from the previous problem, we will have a 3-dimensional range tree with $O(n \log^2 n)$ space complexity. The query time is $O(\log^2 n + k)$.

■